Type-safe Postgres access in cloud-native applications with *sqlc*

Nikolay Kuznetsov

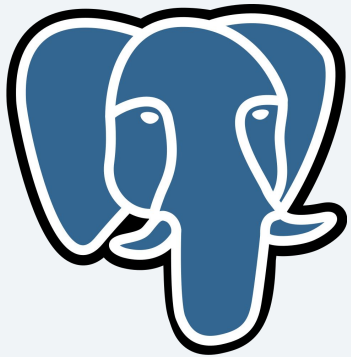
28 May 2026

About me

- Senior Software Engineer
- Zalando Helsinki
- *sqlc* contributor, *Docker* captain
- Author of *pgx-outbox* and *sqlc++* projects

Why Postgres at KCD?

- Open-source, very permissive license
- Kubernetes operators:
  - Zalando (powered by Patroni)
  - CloudNativePg (K8s-native, CNCF)
- Crossplane
  - manages Postgres in external clouds (AWS/GCP/Azure) as K8s custom resources



Why Go at KCD?

- Kubernetes is written in Go
- Cloud-friendly
  - Low memory footprint
  - Fast startup
  - Static binaries

Postgres from Go options

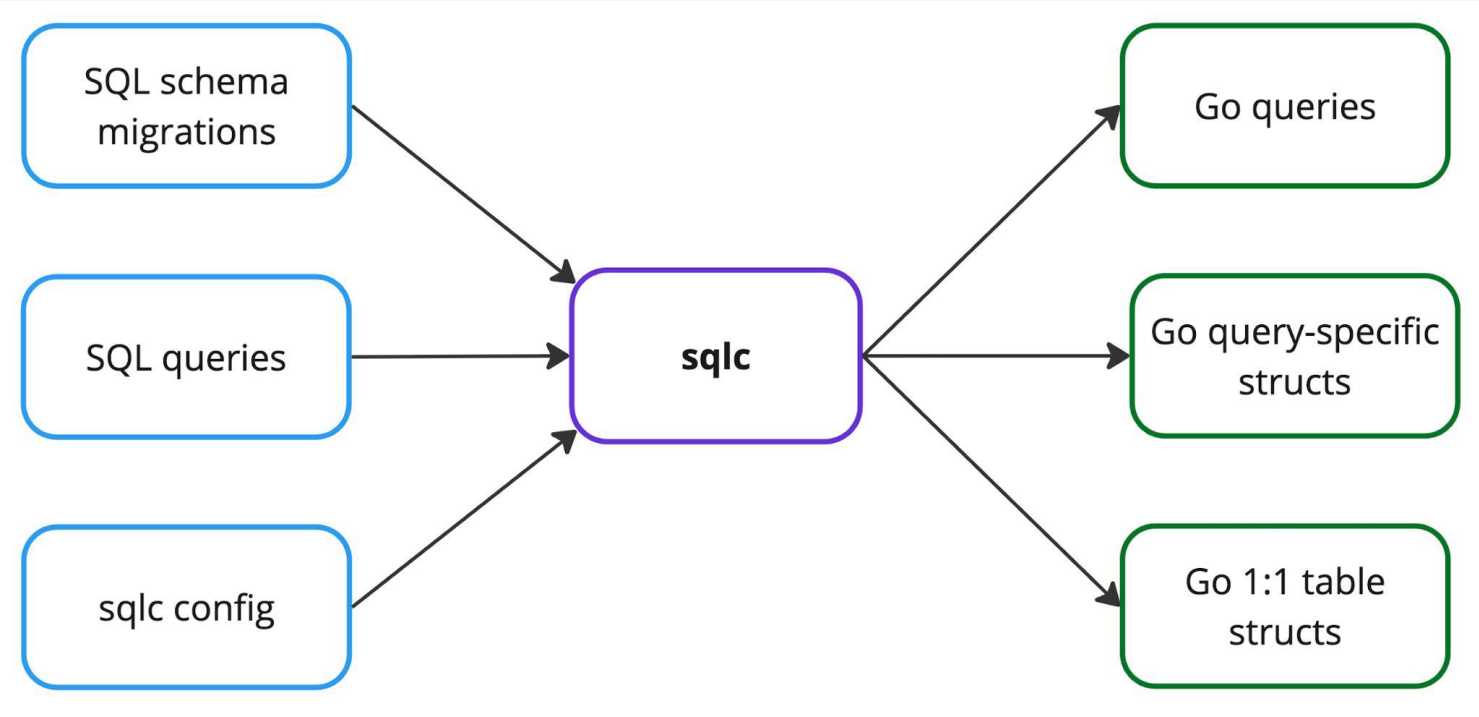
- ORMs (GORM, Ent, Bun)
- **pgx** driver and *pgxpool* API
  - or stdlib *database/sql* API
- *squirrel* to build SQL queries
- **sqlc -** SQL compiler


sqlc

sqlc intro

- Static code generator
- SQL-first approach
- Schema-aware
- Compile-time type safety

sqlc in a nutshell



Cart items table

```
CREATE TABLE IF NOT EXISTS cart_items (

    owner_id        VARCHAR(255)                                NOT NULL,
    product_id      UUID                                        NOT NULL,
    price_amount    DECIMAL                                     NOT NULL,
    price_currency  VARCHAR(3)                                  NOT NULL,

    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,

    PRIMARY KEY (owner_id, product_id)
);
```

Get cart query

```
-- name: GetCart :many
SELECT product_id, created_at
FROM cart_items
WHERE owner_id = $1
```

More options: exec, one, **many**, batchXYZ, etc

Generated record and query

```go
// Code generated by sqlc. DO NOT EDIT.
//   sqlc v1.31.1

type GetCartRow struct {
    ProductID string
    CreatedAt time.Time
}

func (q *Queries) GetCart(ctx, ownerID string) ([]GetCartRow, error) {
    rows, _ := q.db.Query(ctx, "SELECT ...", ownerID)
    defer rows.Close()

    var items []GetCartRow
    for rows.Next() {
        var i GetCartRow
        _ = rows.Scan(&i.ProductID, &i.CreatedAt)
        items = append(items, i)
    }

    return items, nil
}
```

sqlc minimal config

```yaml
version: "2"
sql:
  - engine: "postgresql"
    schema: "internal/migrations"
    queries: "internal/db/queries"
    gen:
      go:
        package: "db"
        out: "internal/db"
        sql_package: "pgx/v5"
```

Pros of *sqlc*

- Less Go boilerplate code to write
  - queries building and execution, rows scanning
- Compile-time schema and type safety
  - uses *Postgres* parser (*wasilibs/go-pgquery*)
  - transforms each query to *AST*
  - builds in-memory catalog (in-memory mode)
- Separation of SQL and Go code

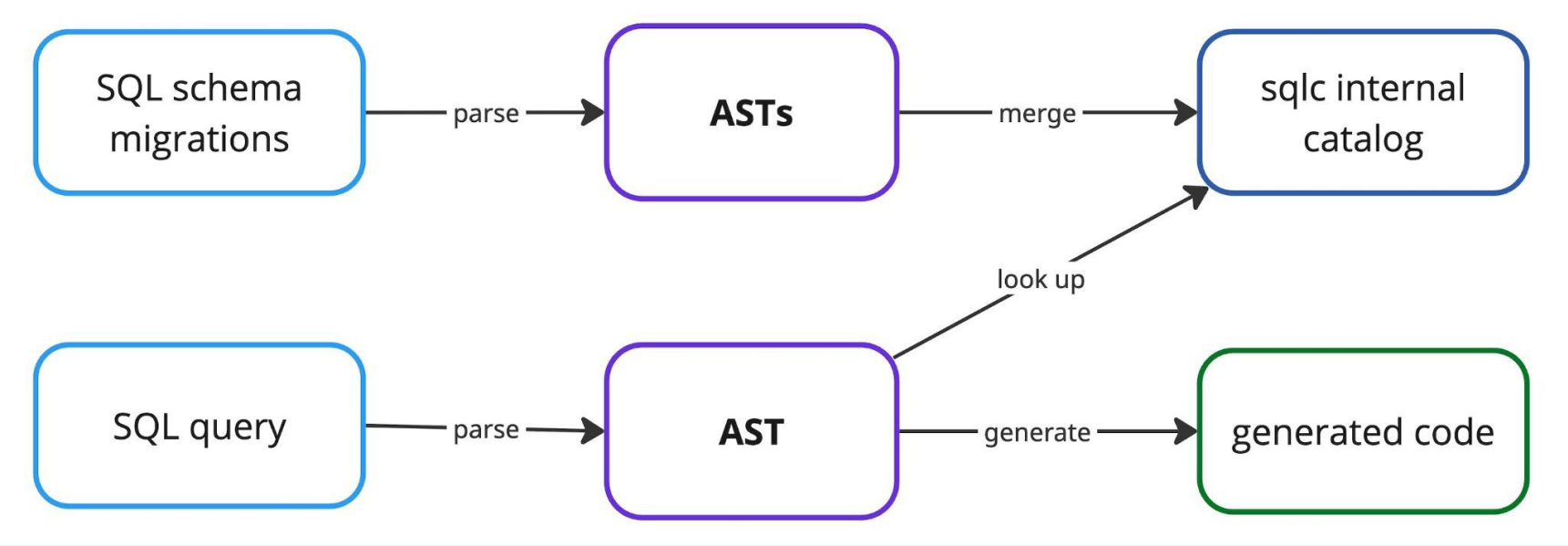
Simplified Abstract Syntax Tree

```
-- name: DeleteItem :execrows
DELETE FROM cart_items WHERE owner_id = $1 AND product_id = $2;
```

```
DeleteStmt
  ├ Relation: cart_items
  └ WhereClause (BoolExpr: AND)
      ├ left: ColumnRef { Name: "owner_id" } = ParamRef{1}
      └ right: ColumnRef { Name: "product_id" } = ParamRef{2}
```

sqlc under the hood

In-memory mode:



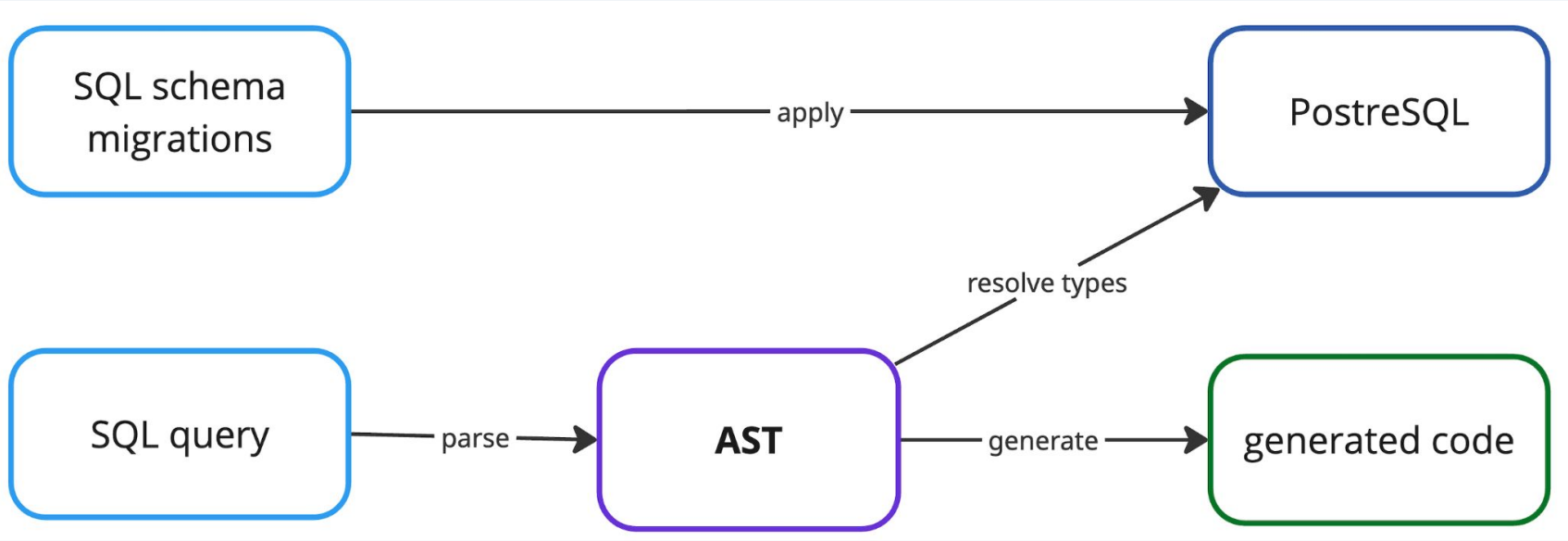
sqlc DB mode config

```
version: "2"
sql:
  - engine: "postgresql"
    database:
      uri: "postgres://user:pass@localhost:5432/sqlc"
    analyzer:
      database: "only"
    schema: "internal/migrations"
    ...
```

Parameter and row types are resolved from DB using extended query protocol: Parse and Describe messages

sqlc under the hood

DB mode:



Challenges with *sqlc*

Potential challenges with *sqlc*

- Dynamic conditional queries
- Generated fields type choice
- Business logic coupling to generated structs
- Batch operations: INSERTs and UPDATEs
- Transaction support

Challenge 1: dynamic queries

- Use case: find all cart items to satisfy the filter

```
type CartFilter struct {
    // optional filter by owner id
    OwnerID    *string

    // optional filter by product ids
    // (OR semantics)
    ProductIDs []uuid.UUID
}
```

Dynamic query options

- Fallback to Go and *squirrel* library API
  - specific query only
  - implement in repository layer
- Keep SQL and *sqlc*, leverage *short-circuiting* technique

squirrel API example

```
q := sq.Select("*").From("cart_items")

if filter.OwnerID != nil {
    q = q.Where(sq.Eq{"owner_id": *filter.OwnerID})
}

if len(filter.ProductIDs) > 0 {
    q = q.Where(sq.Eq{"product_id": filter.ProductIDs})
}

sql, args, _ := q.ToSql()

rows, _ := r.pool.Query(ctx, sql, args...)
```

Short-circuiting

```
-- name: SearchCartItems :many
SELECT *
FROM cart_items
WHERE
  -- short-circuiting: entire condition is TRUE
  -- (filter ignored) when product_ids are NULL
    (@product_ids::UUID[] IS NULL
          OR product_id = ANY(@product_ids))

  -- sqlc.narg() macro generates *string in
  -- SearchCartItemsParams struct
  AND (sqlc.narg(owner_id)::VARCHAR IS NULL
          OR owner_id = sqlc.narg(owner_id));
```

Challenge 2: generated fields type choice

- Default: most accurate database representation: *pgtypes*
- Types can be overridden in *sqlc.yaml*
- Rationale: pick types similar to those used in domain models

```
type GetCartRow struct {
    ProductID     pgtype.UUID
    PriceAmount   pgtype.Numeric
    PriceCurrency string
    CreatedAt     pgtype.Timestamp
}
```

```
type GetCartRow struct {
    ProductID     uuid.UUID
    PriceAmount   decimal.Decimal
    PriceCurrency string
    CreatedAt     time.Time
}
```

Type overrides

```
overrides:
  - db_type: "uuid"
    go_type:
      import: "github.com/google/uuid"
      type: "UUID"
  - db_type: "pg_catalog.timestamp"
    go_type:
      import: "time"
      type: "Time"
  - db_type: "pg_catalog.numeric"
    go_type:
      import: "github.com/shopspring/decimal"
      type: "Decimal"
```

Challenge 3: business logic coupling to *sqlc*-structs

- Avoid using generated *sqlc*-structs in service layer
- Leverage the **Repository** pattern

Repository pattern

- Accepts and returns domain models
- Hides details of SQL, schema, query libraries
- Maps between DB records and domain models
- Orchestrates transactions across multiple operations

Domain cart

```
type Cart struct {
    OwnerID string
    Items   []CartItem
}

type CartItem struct {
    ProductID uuid.UUID // google/uuid
    Price     Money

    CreatedAt time.Time
}

type Money struct {
    Amount   decimal.Decimal   // shopspring/decimal
    Currency currency.Unit     // x/text/currency
}
```

Cart repository interface

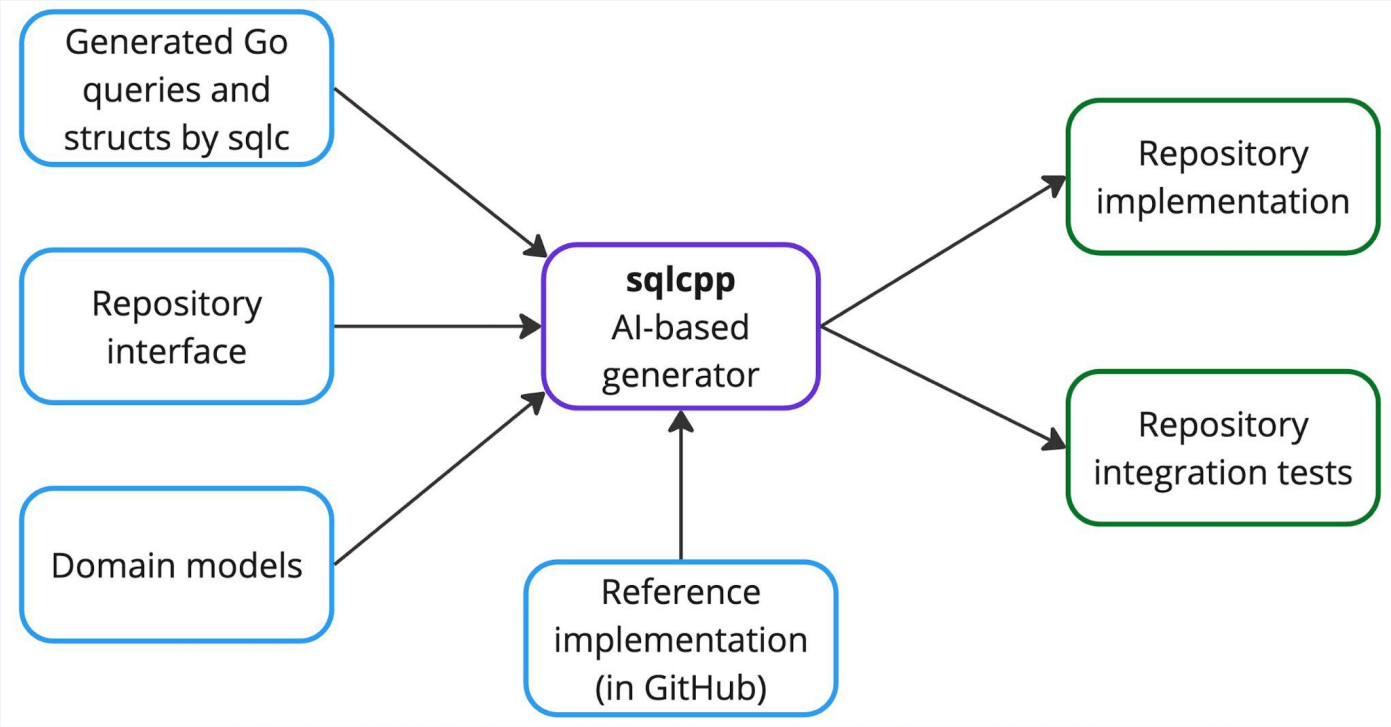
```
type CartRepository interface {
    // context.Context type is omitted for brevity

    GetCart(ctx, ownerID string) (domain.Cart, error)

    AddItem(ctx, ownerID string, item domain.CartItem) error

    DeleteItem(ctx, ownerID string, productID uuid.UUID) (bool, error)
}
```

Delegating to generated queries

```go
func (r *repo) GetCart(ctx context.Context, ownerID string) (domain.Cart, error) {

    // rows []GetCartRow - generated by sqlc
    // q *db.Queries - generated by sqlc
    rows, _ := r.q.GetCart(ctx, ownerID)

    // map sqlc rows -> domain items
    items := mapGetCartRowsToDomain(rows)


    return domain.Cart{OwnerID: ownerID, Items: items}, nil

}
```

Create coding agent skills to do delegation and mapping

sqlc++ in a nutshell



Challenge 4: Batch operations challenge

- Use case: INSERT / UPDATE multiple rows efficiently
- Leverage ***:batchexec*** and ***:batchmany*** query annotations
- Code with ***pgx.Batch*** gets generated
- *pgx* uses prepared statements and Postgres pipeline mode

Batch INSERT example

```
-- name: AddItemsBatch :batchexec
INSERT INTO cart_items
    (owner_id, product_id, price_amount, price_currency)
VALUES ($1, $2, $3, $4)
ON CONFLICT (owner_id, product_id) DO NOTHING;
```

Generated pgx.Batch code

```
batch := &pgx.Batch{}

// arg []AddItemsBatchParams
for _, a := range arg {
    // set vals from a
    batch.Queue(AddItemsBatch, vals...)
}

// pgx.BatchResults
return q.db.SendBatch(ctx, batch)
```

Challenge 5: Transaction support

- Use case: run multiple *sqlc*-queries in a tx
- Use case: run multiple repo methods in a tx
- Leverage *DTBX* interface generated by *sqlc*

DBTX interface

```
// common interface between pgxpool.Pool and pgx.Tx
type DBTX interface {
    Exec(ctx, string, ...interface{}) (pgconn.CommandTag, error)
    Query(ctx, string, ...interface{}) (pgx.Rows, error)
    QueryRow(ctx, string, ...interface{}) pgx.Row
    // only if batch annotations are used
    SendBatch(ctx, *pgx.Batch) pgx.BatchResults
}
```

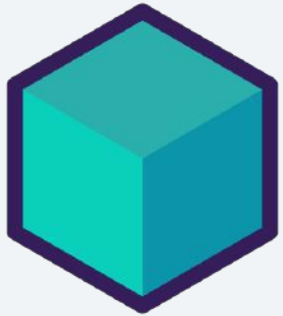
withTx helper in repository layer

```
err := withTx(ctx, r.dbtx, func(q *db.Queries) error {

    if err := q.AddItem(ctx, addParams); err != nil {
        return err
    }

    if err := q.DeleteItem(ctx, deleteParams); err != nil {
        return err
    }

    return nil
})
```

withTx internals

```go
// withTx executes fn within a transaction if dbtx is a pool,
// or uses the existing transaction if dbtx is already a transaction.

func withTx(ctx, dbtx db.DBTX, fn func(q *db.Queries) error) error {

    if tx, ok := dbtx.(pgx.Tx); ok {
        return fn(db.New(tx))
    }

    // pool, ok := dbtx.(*pgxpool.Pool)
    // pool.Begin, fn(db.New(tx)), tx.Commit/Rollback logic
```

Testing repositories



- Testcontainers + Postgres module
- Suite from *stretchr/testify*
- Table tests for each repository method
- Helpful libs: *gofakeit*, *go-cmp*

Takeaways

- Adopt *sqlc*, prefer DB-mode
- Avoid *sqlc*-generated structs in business logic
- Do not ignore transactions
- Repository boilerplate can be automated

Thank you!

Q&A

Nikolay Kuznetsov

nikolayk812 / sqlcpp

nkuznetsov